

Vector Databases and Retrieval Systems for Large-Scale AI Applications

Abstract

The proliferation of large-scale artificial intelligence (AI) applications, particularly those leveraging dense vector embeddings for semantic search, Retrieval-Augmented Generation (RAG), and multi-modal reasoning, has necessitated a paradigm shift in data management. Traditional database systems, optimized for exact match queries on structured data, are ill-suited for the high-dimensional, approximate nearest neighbor search (ANNS) requirements of modern AI workloads. This survey provides a comprehensive analysis of the state-of-the-art in vector databases and retrieval systems, synthesizing insights from 300 recent academic contributions. We present a unified taxonomy of methodological families, ranging from graph-based indexing and product quantization to learned indices and hardware-software co-design. We critically examine the evolution from memory-resident static indices to distributed, disk-resident, and cloud-native architectures capable of handling billion-scale datasets. Furthermore, we explore specialized retrieval paradigms including filtered search, multi-vector retrieval, and privacy-preserving protocols. By evaluating trade-offs in latency, throughput, memory footprint, and update efficiency, this paper identifies key bottlenecks in current systems and outlines future research directions in agentic retrieval, generative indexing, and sustainable infrastructure design.

1. Introduction

The integration of deep learning into information retrieval has transformed unstructured data—text, images, audio, and video—into high-dimensional vector spaces where semantic similarity is encoded as geometric proximity [1], [2]. Consequently, the core operation in AI-driven applications has become the Approximate Nearest Neighbor Search (ANNS): given a query vector, efficiently retrieving the k most similar vectors from a massive corpus [3], [4]. While early solutions relied on Locality-Sensitive Hashing (LSH) and tree-based structures, the “curse of dimensionality” rendered these methods ineffective for modern embeddings, leading to the dominance of graph-based indices like HNSW and quantization-based approaches like Product Quantization (PQ) [5], [3], [6].

The scale of these applications has grown exponentially, moving from million-scale benchmarks to billion-scale production systems serving real-time queries for recommendation engines, genomics, and legal discovery [7], [8], [9]. This scaling has exposed fundamental limitations in traditional in-memory architectures, driving innovation in out-of-core indexing, distributed storage, and heterogeneous hardware utilization [10], [11], [12]. Simultaneously, the rise of Retrieval-Augmented Generation (RAG) has introduced new constraints, requiring retrieval systems to support complex filtering, dynamic updates, and multi-stage refinement pipelines to ensure factual grounding and reduce hallucination [13], [14], [15].

This survey synthesizes the rapidly expanding literature on vector database management systems (VDBMS) and retrieval algorithms. Unlike prior works that focus narrowly on algorithmic accuracy or specific system implementations, we provide a holistic view covering the entire stack: from low-level distance computation optimizations and index construction algorithms to high-level architectural patterns for cloud-native deployment and agentic workflows. We categorize methods based on their underlying geometric principles, storage strategies, and optimization targets, drawing evidence from a diverse set of 300 papers spanning 2019 to 2027. Our analysis reveals a clear trend toward hybridization: combining the navigability of graphs with the compression of quantization,

the scalability of distributed systems with the efficiency of near-data processing, and the rigidity of static indices with the adaptability of learned models.

2. Method

The landscape of vector retrieval is characterized by a tension between accuracy, latency, memory consumption, and scalability. To navigate this complexity, we organize the field into coherent methodological families. We first examine the foundational algorithmic paradigms of graph-based and quantization-based search. We then explore architectural innovations for scaling beyond single-node memory limits, followed by specialized techniques for filtered search, dynamic updates, and multi-vector retrieval. Finally, we discuss the integration of machine learning into indexing and the emerging domain of privacy-preserving and access-controlled retrieval.

2.1 Graph-Based Approximate Nearest Neighbor Search

Graph-based indices have emerged as the dominant paradigm for high-recall, low-latency ANNS, leveraging the “small-world” phenomenon to navigate high-dimensional spaces efficiently.

2.1.1 Hierarchical Navigable Small World (HNSW) and Variants The Hierarchical Navigable Small World (HNSW) algorithm remains the gold standard for in-memory vector search, constructing a multi-layered graph where upper layers provide long-range navigation and lower layers offer fine-grained precision [16], [1]. Recent work has focused on optimizing HNSW construction and traversal. *FlatNav* challenges the necessity of hierarchy in high dimensions, proposing that the bottom layer alone forms a sufficient “hub highway” due to the natural emergence of hub nodes, reducing memory consumption by nearly 40% while maintaining latency [17]. Conversely, *HNSW++* introduces a dual-branch structure and Local Intrinsic Dimensionality (LID)-driven insertion to mitigate local minima, improving recall by up to 30% on computer vision tasks [18].

Optimization of the search process itself has also seen significant advances. *Speed-ANN* relaxes the strict best-first ordering of node expansion to enable intra-query parallelism, achieving up to 76x latency reduction on multi-core systems by avoiding synchronization barriers [19]. *CRouting* reduces the computational cost of distance calculations during traversal by estimating neighbor distances via the Cosine Theorem and intrinsic angle distributions, yielding a 1.48x speedup with negligible recall loss [20]. Furthermore, *AQR-HNSW* integrates density-aware adaptive quantization, allocating fewer bits to sparse regions and more to dense clusters, achieving a 3x throughput improvement over baseline HNSW [21].

2.1.2 Disk-Resident and Out-of-Core Graph Indices As datasets exceed DRAM capacity, disk-resident graph indices have become critical. *DiskANN* (Vamana) pioneered the use of SSDs for billion-scale search by storing vectors on disk while keeping a compressed graph in memory [8]. Subsequent works have refined I/O patterns and layout strategies. *PageANN* aligns logical graph nodes with physical SSD pages, eliminating partial page reads and achieving 7x higher throughput than SPANN at equivalent recall [22]. *Gorgeous* decouples adjacency list storage from vector data, prioritizing the caching of frequently accessed neighbor lists to boost query throughput by 60% [23].

To further reduce I/O latency, *GateANN* introduces “graph tunneling,” traversing non-matching nodes entirely in DRAM using approximate distances and only fetching full-precision vectors for promising candidates, reducing SSD I/O by 10.2x [24]. *LAANN* employs a look-ahead strategy

that dynamically adjusts beam width between approach and convergence phases, co-optimizing CPU and I/O pipelines to achieve 4x higher throughput than PipeANN [25]. *VeloANN* utilizes a coroutine-based asynchronous runtime to overlap SSD I/O with distance computation, eliminating CPU idle time and delivering 5.8x higher throughput than DiskANN [26].

2.1.3 Construction and Maintenance Optimization Constructing high-quality graphs at scale is computationally expensive. *PiPNN* replaces incremental search with batched distance computation via optimized GEMM kernels and overlapping partitions, accelerating construction by 11.6x on billion-scale datasets [27]. *ParlayANN* introduces lock-free parallelism using prefix-doubling and work-stealing schedulers, enabling deterministic batch insertion without per-vertex locks [28]. For dynamic environments, *FreshDiskANN* separates recent updates into an in-memory temporary index merged periodically with the disk-resident long-term index, maintaining >95% recall under continuous updates [29]. *SPatch* reframes deletion as a random walk sparsification problem, using star-mesh transforms to preserve hitting times and navigability after node removal [30].

2.2 Quantization and Compression Techniques

Vector quantization reduces memory footprint and accelerates distance computation by representing high-dimensional vectors with compact codes.

2.2.1 Product Quantization (PQ) and Variants Product Quantization (PQ) decomposes vectors into subvectors quantized independently, enabling efficient asymmetric distance computation [3]. *RaBitQ* and *IVF-RaBitQ* extend this by using binary quantization with theoretical error bounds, enabling GPU-native pipelines that construct indices 7.7x faster than CAGRA while maintaining high recall [31]. *LeanVec* employs query-aware linear dimensionality reduction combined with lattice vector quantization (LVQ), reducing memory footprint without degrading graph topology quality, achieving 10.2x throughput improvements [32]. *Locally-adaptive Vector Quantization (LVQ)* applies per-vector mean-centering and dynamic range scaling to maximize bit utilization, enabling aggressive 4-8 bit compression with minimal accuracy loss [33].

2.2.2 Scalar and Non-Uniform Quantization Beyond product quantization, scalar methods offer flexibility. *SQUASH* utilizes Optimized Scalar Quantization (OSQ) with shared segment-based storage to compress vectors and attributes into power-of-two bit segments, achieving 18x throughput improvement in serverless environments [34]. *Individualized Non-Uniform Quantization (NVQ)* fits unique non-linear quantizers per vector using gradient-free optimization, reducing storage footprint by 3.44x while preserving high-recall performance [35]. *QuIVer* constructs graph topologies directly in a 2-bit sign-magnitude binary quantized metric space, achieving 15.6x higher throughput than hnswlib on cosine-native datasets [36].

2.2.3 Learned and Adaptive Compression Learned methods adapt compression to data distributions. *AdANNS* leverages Matryoshka representations to assign varying dimensionalities to different search stages, reducing compute and storage costs by half while improving accuracy [37]. *Game-Theoretic Latent Compression* models the interaction between an encoder and retriever as a zero-sum game, achieving an equilibrium that balances storage efficiency with semantic retrieval accuracy [38]. *FaTRQ* introduces tiered residual quantization, storing coarse codes in fast memory and compact ternary residuals in far memory (CXL), enabling progressive distance estimation with 9.4x throughput speedup [39].

2.3 Scalable Architectures: Distributed and Cloud-Native Systems

Scaling to billions of vectors requires distributing data across multiple nodes or leveraging heterogeneous storage tiers.

2.3.1 Distributed Graph Traversal Distributed systems must balance load and minimize network overhead. *DISTRIBUTEDANN* abstracts distributed storage as a single logical disk, duplicating compressed vectors into graph nodes to eliminate extra network hops, achieving >100,000 QPS on a 50B vector index [40]. *BatANN* employs asynchronous state-passing, transferring the full query state to remote servers rather than fetching data, halving communication latency [41]. *Co-Tra* differentiates execution modes based on partition locality, using synchronized local queues for primary machines and pull-push modes for secondary machines, achieving 9.8–13.4x speedup over single-machine baselines [42]. *Harmony* integrates vector-based and dimension-based partitioning with a cost model to dynamically balance load, improving throughput by 4.63x [43].

2.3.2 Cloud-Native and Disaggregated Storage Cloud-native architectures decouple compute and storage to optimize cost and elasticity. *d-HNSW* constructs a lightweight meta-index locally for routing while storing sub-indices in remote RDMA-accessible memory, reducing latency by 117x compared to naive disaggregated approaches [44], [45]. *Attaching Approximate Nearest Neighbor Indexes to Apache Iceberg Snapshots* binds ANN index lifecycles to table snapshots, enabling transactional guarantees and tiered probing strategies for billion-scale indices [46]. *Cloud-Native Vector Search* analyses trade-offs between cluster and graph indexes under remote storage constraints, finding that graph indexes scale better in high-concurrency workloads despite higher roundtrip counts [47]. *ScaleGANN* utilizes cost-effective cloud spot GPUs for shard construction and CPU-based merging, achieving 9x faster index build times at 6x lower cost [48].

2.3.3 Heterogeneous and Second-Tier Memory Emerging hardware technologies offer new scaling avenues. *Characterizing the Dilemma of Performance and Index Size* demonstrates that second-tier memory (RDMA/CXL/NVM) supports fine-grained access patterns ideal for vector indices, outperforming cluster indices by reading 95–97% fewer bytes [49]. *COSMOS* offloads the entire search pipeline to programmable cores within CXL memory controllers, eliminating host bottlenecks and achieving 6.72x higher throughput [50]. *HAVEN* replaces partial HBM stacks with High-Bandwidth Flash (HBF) and integrates near-storage search units, enabling full database residency with 20x throughput improvement over GPU-SSD systems [51].

2.4 Filtered and Hybrid Search

Real-world applications often require combining vector similarity with scalar filters (e.g., date ranges, categories).

2.4.1 Pre-Filtering and Post-Filtering Strategies Naive pre-filtering (scanning only matching vectors) suffers from low selectivity, while post-filtering (scanning all vectors then filtering) wastes computation. *ACORN* modifies HNSW construction by expanding neighbor lists to ensure predicate subgraph connectivity, outperforming pre/post-filtering by three orders of magnitude [52]. *FAVOR* employs a selectivity-aware dual strategy, using brute-force for low selectivity and inline filtering for others, dynamically reshaping vector distributions via exclusion distances [53]. *Elastic Index Select (ELI)* defines an “elastic factor” based on label overlap to guide greedy index selection, achieving 800x efficiency speedups in label-hybrid search [54].

2.4.2 Geometric Transformation and Fusion Advanced methods fuse attributes and vectors into a unified space. *FusedANN* applies a convex transformation to fuse attribute vectors into the content space, enabling standard ANN algorithms to handle complex predicates without index modification [55]. *Filter-Centric Vector Indexing (FCVI)* partitions vectors and subtracts scaled filter vectors, clustering similar filters while preserving intra-cluster similarity, resulting in 3x faster latency [56]. *Beta-Window Search Tree (beta-WST)* constructs a balanced tree over label-sorted data with Vamana indices at internal nodes, achieving 75x speedup over baselines for window filters [57].

2.4.3 GPU-Accelerated Filtering *VecFlow* partitions data via label-centric Inverted File (IVF) indexing, routing high-specificity labels to redundancy-bypassing virtual graph indices on GPUs, achieving 5 million QPS at 90% recall [58]. *GRAB-ANNS* partitions vectors into disjoint predicate buckets, transforming range filtering into lightweight bucket selection with a hybrid edge topology, yielding 240x speedup on GPU [59].

2.5 Multi-Vector and Set-Based Retrieval

Multi-vector models (e.g., ColBERT) represent documents as sets of token embeddings, requiring specialized retrieval mechanisms.

2.5.1 Late Interaction and MaxSim Approximation *LEMUR* reformulates MaxSim calculation as a supervised multi-output regression problem, mapping token embeddings to a latent space where inner products approximate MaxSim, enabling 5-11x faster search [60]. *MUVERA* transforms multi-vector sets into Fixed Dimensional Encodings (FDEs) via randomized space partitioning, reducing Chamfer similarity search to single-vector MIPS with 10% higher recall than PLAID [61]. *ConstBERT* projects variable token counts to a fixed set of learned document-level vectors, achieving comparable effectiveness to ColBERT with >50% index size reduction [62].

2.5.2 Graph and Clustering Approaches *GEM* constructs a set-level dual-graph using Earth Mover’s Distance for navigation and Chamfer distance for scoring, stabilizing topology without sacrificing accuracy [63]. *Tachiom* employs token-aware clustering to partition centroid allocation by token frequency, reducing clustering complexity from $O(N \cdot k)$ to linear and achieving 9.8x lower query latency [64]. *No More K-means* replaces K-means clustering with neuron-level inverted indexing via Sparse Autoencoders, enabling direct sparse inverted indexing 15x faster than ColBERTv2 [65].

2.6 Learned Indices and AI-Optimized Search

Machine learning is increasingly used to model data distributions and optimize search paths.

2.6.1 Learning-to-Index and Routing *LIRA* employs a probing model to predict partition probabilities based on query vectors and centroid distances, eliminating irrelevant partition probing and reducing distance computations [66]. *Storage Neural Network Augmented ANN* uses a three-layer MLP to predict optimal clusters for storage-resident vectors, bypassing standard centroid-distance calculations and reducing vectors fetched by 80% [67]. *PivNet* leverages pre-computed k-NN distances of spatially proximate pivots as features for a neural network to estimate k-NN distances, achieving 17.3x inference speedup [68].

2.6.2 AI-Driven Optimization and Tuning *LLM-Guided ANN Index Optimization* utilizes phase-aware LLM agents to navigate coupled parameter spaces (engine, pool size, fusion weight), outperforming traditional hyperparameter optimization by 33% [69]. *EnhanceGraph* constructs a conjugate graph using historical search logs to recover pruned edges and inject routing paths, improving Recall@1 from 41.74% to 93.42% [70]. *Adaptive Prefiltering* exploits a power-law relationship between training frequency and cluster coherence to allocate heterogeneous probing budgets, reducing search costs by 20.4% [71].

2.7 Privacy, Security, and Access Control

As vector databases store sensitive data, privacy-preserving and access-controlled retrieval are critical.

2.7.1 Privacy-Preserving Search *SP-A²NN* constructs a sharable bitgraph structure replacing undirected HNSW graphs to enable linear complexity sharing via Shamir’s secret sharing, protecting raw vectors during collaborative RAG [72]. *Distance Comparison Encryption (DCE)* enables secure exact distance comparisons via vector randomization, combining approximate DCPE-based indexing with exact DCE-based refinement for a 100-1000x speedup over fully homomorphic baselines [73]. *PIR-RAG* partitions corpora into semantic clusters and retrieves entire clusters via a single homomorphic matrix-vector product, amortizing PIR costs while maintaining NDCG@10 of 0.799 [74].

2.7.2 Access Control and Role-Based Filtering *HoneyBee* formulates partitioning as a Mixed-Integer Nonlinear Programming (MINLP) problem to minimize latency under Role-Based Access Control (RBAC) constraints, achieving 6x latency speedup over row-level security [75]. *Veda* and *EffVeda* decompose datasets into disjoint exclusive blocks organized in an access-aware lattice, optimizing storage-query trade-offs via targeted copying and merging [76].

2.8 Evaluation Metrics and Benchmarks

Accurate evaluation is paramount. *1/Ratio@k* proposes an inverse approximation ratio metric that tracks downstream RAG quality better than traditional Recall@k, requiring fewer distance computations [77]. *Vector Index Benchmark for Embeddings (VIBE)* systematically includes Out-of-Distribution (OOD) datasets to evaluate robustness, revealing that graph-based methods suffer significant degradation on OOD queries unless specifically tuned [78]. *FANNBench* provides a unified interface for evaluating 12 filtering algorithms across range, label, and arbitrary categories, highlighting that segmented subgraph indexing outperforms edge-segmentation in low-selectivity queries [79].

3. Challenges and Outlook

Despite significant progress, several critical challenges remain in the development of vector databases and retrieval systems.

3.1 The Dynamic Update Bottleneck

While static indexing is well-understood, supporting high-frequency updates (insertions and deletions) without degrading query performance remains difficult. Graph-based indices like HNSW suffer from “unreachable points” and topology degradation upon deletion [80]. Although methods

like *FreshDiskANN* [29] and *LSM-VEC* [81] introduce LSM-tree-like structures or temporary indices to handle updates, they often incur latency spikes during merge phases or require complex background consolidation. Future research must focus on truly online algorithms that maintain graph navigability and quantization quality in real-time streaming scenarios without periodic rebuilds.

3.2 Out-of-Distribution (OOD) Robustness

Most ANNS algorithms assume query distributions similar to the training data. However, in dynamic AI applications, queries often drift or originate from OOD distributions. *VIBE* [78] and *LeanVec* [32] highlight that many SotA methods degrade significantly under OOD conditions. Learned indices, which rely heavily on distributional assumptions, are particularly vulnerable [67], [66]. Developing distribution-agnostic or adaptive indices that can detect and adjust to distribution shifts autonomously is a crucial open problem.

3.3 Evaluation and Benchmarking Gaps

Current benchmarks often focus on static recall-throughput trade-offs on clean datasets. There is a lack of standardized benchmarks for filtered search, multi-vector retrieval, and dynamic workloads. *FANNBench* [79] and *arxiv-for-fanns* [82] are steps forward, but the field lacks comprehensive suites that evaluate end-to-end RAG performance, including latency, cost, and factual accuracy under realistic load. Furthermore, the reliance on “Recall@k” as the primary metric is increasingly questioned, with proposals like $1/\text{Ratio}@k$ [77] suggesting that geometric proximity is a better proxy for downstream task quality.

3.4 Energy Efficiency and Sustainability

As vector databases scale to billions of vectors, energy consumption becomes a major concern. GPU-accelerated and near-data processing solutions like *HAVEN* [51] and *NDSEARCH* [83] offer efficiency gains, but the carbon footprint of large-scale indexing and continuous re-training of learned indices remains under-studied. Future systems must incorporate energy-aware scheduling and hardware selection, potentially leveraging specialized low-power accelerators or optimizing for “green” retrieval paths.

3.5 Agentic and Generative Retrieval

The rise of Agentic RAG introduces new requirements for retrieval systems. Agents may need to perform multi-step reasoning, iterative refinement, and tool-augmented search [84], [85]. Traditional “retrieve-once” paradigms are insufficient. Systems must support interactive sessions, context-aware caching, and dynamic index modification based on agent feedback. Moreover, *Generative Retrieval* methods that replace explicit indices with autoregressive generation of document identifiers [86], [87] challenge the very definition of a vector database, suggesting a future where retrieval and generation are fully unified.

3.6 Security and Trust

As vector databases become central to enterprise AI, security threats such as model inversion, membership inference, and prompt injection via retrieved context grow [15]. While cryptographic solutions like *SP-A²NN* [72] and *DCE* [73] exist, they often incur prohibitive overhead. Developing practical, low-latency security mechanisms that protect both the data and the query without

compromising the utility of the retrieval system is a critical frontier. Additionally, ensuring the trustworthiness of retrieved content against adversarial poisoning attacks remains an open challenge.

4. Conclusion

The field of vector databases and retrieval systems has evolved rapidly from simple in-memory indices to sophisticated, distributed, and hardware-aware ecosystems. This survey has synthesized 300 academic contributions to present a comprehensive view of the state-of-the-art. We observe a clear convergence toward hybrid architectures that combine the strengths of graph navigation, quantization compression, and learned optimization. The transition from static, single-node systems to dynamic, cloud-native, and disaggregated architectures reflects the growing demands of large-scale AI applications.

However, significant challenges persist. The tension between update efficiency and query performance, the robustness of indices against distribution shifts, and the need for standardized, holistic evaluation frameworks remain unresolved. As AI applications become more agentic and generative, retrieval systems must evolve from passive data stores to active participants in the reasoning loop. Future research will likely focus on unifying retrieval and generation, enhancing security and privacy guarantees, and optimizing for sustainability. By addressing these challenges, the next generation of vector databases will not only support but actively accelerate the deployment of trustworthy and scalable AI systems.

References

- [1] James Jie Pan, Jianguo Wang, Guoliang Li. (2024). Survey of Vector Database Management Systems. arXiv:2310.14021.
- [2] Le Ma, Ran Zhang, Yikun Han. (2024). A Comprehensive Survey on Vector Database: Storage and Retrieval Technique, Challenge. arXiv:2310.11703.
- [3] Matthijs Douze, Alexandr Guzhva, Chengqi Deng. (2024). THE FAISS LIBRARY. arXiv:2401.08281.
- [4] Yifan Wang. (2024). A Survey on Efficient Processing of Similarity Queries over Neural Embeddings. arXiv:2204.07922.
- [5] Ilias Azizi, Karima Echihabi, Themis Palpanas. (2025). Toward Efficient and Scalable Design of In-Memory Graph-Based Vector Search. arXiv:2509.05750.
- [6] Mengzhao Wang, Xiaoliang Xu, Qiang Yue. (2021). A Comprehensive Survey and Experimental Comparison of Graph-Based Approximate Nearest Neighbor Search. arXiv:2101.12631.
- [7] Harsha Vardhan Simhadri, George Williams, Martin Aumüller. (2021). Results of the NeurIPS’21 Challenge on Billion-Scale Approximate Nearest Neighbor Search. arXiv:2205.03763.
- [8] Kun Yu, Jiabao Jin, Xiaoyao Zhong. (2024). Approximate Nearest Neighbor Search of Large Scale Vectors on Distributed Storage. arXiv:2510.17326.
- [9] Simeon Emanuilov, Aleksandar Dimov. (2024). Billion-Scale Similarity Search Using a Hybrid Indexing Approach with Advanced Filtering. arXiv:2501.13442.

- [10] Kento Tatsuno, Daisuke Miyashita, Taiga Ikeda. (2024). AiSAQ : All-in-Storage ANNS with Product Quantization for DRAM-free Information Retrieval. arXiv:2404.06004.
- [11] Xiaoyu Chen, Jinxiu Qu, Yitong Song. (2024). Disk-Resident Graph ANN Search: An Experimental Evaluation. arXiv:2603.01779.
- [12] Yitong Song, Xuanhe Zhou, Christian S. Jensen. (2025). Vector Search for the Future: From Memory-Resident, Static Heterogeneous Storage, to Cloud-Native Architectures. arXiv:2601.01937.
- [13] Santosh Bhupathi. (2024). Role of Databases in GenAI Applications. arXiv:2503.04847.
- [14] Amirreza Abbasi, Mohsen Hooshmand. (2024). Beyond Pipelines: A Fundamental Study on the Rise of Generative-Retrieval Architectures in Web Research. arXiv:2602.17450.
- [15] Dean Wampler, Dave Nielson, Alireza Seddighi. (2025). Engineering the RAG Stack: A Comprehensive Review of the Architecture and Trust Frameworks for Retrieval Augmented Generation Systems. arXiv:2601.05264.
- [16] Gulshan Yadav, RahulKumar Yadav, Mansi Viramgama. (2024). Quantixar: High-Performance Vector Data Management System. arXiv:2403.12583.
- [17] Blaise Munyampirwa, Vihan Lakshman, Benjamin Coleman. (2024). Down with the Hierarchy: The 'H' in HNSW Stands for "Hubs". arXiv:2412.01940.
- [18] Hy Nguyen, Nguyen Hung Nguyen, Nguyen Linh Bao Nguyen. (2024). DUAL-BRANCH HNSW APPROACH WITH SKIPBRIDGES AND LID-DRIVEN OPTIMIZATION. arXiv:2501.13992.
- [19] Zhen Peng, Minjia Zhang, Kai Li. (2024). Speed-ANN: Low-Latency and High-Accuracy Nearest Neighbor Search via Intra-Query Parallelism. arXiv:2201.13007.
- [20] Zhenxin Li, Shuibing He, Jiahao Guo. (2024). CRouting: Reducing Expensive Distance Calls in Graph-Based Approximate Nearest Neighbor Search. arXiv:2509.00365.
- [21] Ganap Ashit Tewary, Nrusinga Charan Gantayat, Jie Zhang. (2024). AQR-HNSW: Accelerating Approximate Nearest Neighbor Search via Density-aware Quantization and Multi-stage Re-ranking. arXiv:2602.21600.
- [22] Dingyi Kang, Dongming Jiang, Hanshen Yang. (2024). Scalable Disk-Based Approximate Nearest Neighbor Search with Page-Aligned Graph. arXiv:2509.25487.
- [23] Peiqi Yin, Xiao Yan, Qihui Zhou. (2024). Gorgeous: Revisiting the Data Layout for Disk-Resident High-Dimensional Vector Search. arXiv:2508.15290.
- [24] Nakyoung Lee, Soobin Cho, Jiwoong Park. (2024). GateANN: I/O-Efficient Filtered Vector Search on SSDs. arXiv:2603.21466.
- [25] Dingyi Kang, Juncheng Yang, Bingzhe Li. (2024). LAANN: I/O-Aware Look-Ahead Search for Disk-Based Approximate Nearest Neighbor Search. arXiv:2606.02784.
- [26] Weichen Zhao, Yuncheng Lu, Yao Tian. (2026). Optimizing SSD-Resident Graph Indexing for High-Throughput Vector Search. arXiv:2602.22805.
- [27] Tobias Rubel, Richard Wen, Laxman Dhulipala. (2024). PiPNN: Ultra-Scalable Graph-Based Nearest Neighbor Indexing. arXiv:2602.21247.

- [28] Magdalen Dobson Manohar, Zheqi Shen, Guy E. Blelloch. (2024). ParlayANN: Scalable and Deterministic Parallel Graph-Based Approximate Nearest Neighbor Search Algorithms. arXiv:2305.04359.
- [29] Aditi Singh, Suhas Jayaram Subramanya, Ravishankar Krishnaswamy. (2024). FreshDiskANN: A Fast and Accurate Graph-Based ANN Index for Streaming Similarity Search. arXiv:2105.09613.
- [30] Nina Mishra, Yonatan Naamad, Tal Wagner. (2025). Graph-based Nearest Neighbors with Dynamic Updates via Random Walks. arXiv:2512.18060.
- [31] Jifan Shi, Jianyang Gao, James Xia. (2024). GPU-Native Approximate Nearest Neighbor Search with IVF-RaBitQ: Fast Index Build and Search. arXiv:2602.23999.
- [32] Mariano Tepper, Ishwar Singh Bhati, Cecilia Aguerrebere. (2024). LeanVec: Searching vectors faster by making them fit. arXiv:2312.16335.
- [33] Cecilia Aguerrebere, Ishwar Singh Bhati, Mark Hildebrand. (2024). Similarity search in the blink of an eye with compressed indices. arXiv:2304.04759.
- [34] Joe Oakley, Hakan Ferhatosmanoglu. (2024). SQUASH: Serverless and Distributed Quantization-based Attributed Vector Similarity Search. arXiv:2502.01528.
- [35] Mariano Tepper, Ted Willke. (2024). Individualized non-uniform quantization for vector search. arXiv:2509.18471.
- [36] Wenxuan Xiao, Zhiyou Wang, Chengcheng Li. (2024). QuIVer: Rethinking ANN Graph Topology via Training-Free Binary Quantization. arXiv:2605.02171.
- [37] Aniket Rege, Aditya Kusupati, Sharan Ranjit. (2024). AdANNS: A Framework for Adaptive Semantic Search. arXiv:2305.19435.
- [38] Kushagra Agrawal, Nisharg Nargund, Oishani Banerjee. (2024). Optimization of Latent-Space Compression using Game-Theoretic Techniques for Transformer-Based Vector Search. arXiv:2508.18877.
- [39] Tianqi Zhang, Flavio Ponzina, Tajana Rosing. (2024). FaTRQ: Tiered Residual Quantization for LLM Vector Search in Far-Memory-Aware ANNS Systems. arXiv:2601.09985.
- [40] Philip Adams, Menghao Li, Shi Zhang. (2024). DISTRIBUTEDANN: Efficient Scaling of a Single DISKANN Graph Across Thousands of Computers. arXiv:2509.06046.
- [41] Nam Anh Dang, Ben Landrum, Ken Birman. (2020). Passing the Baton: High Throughput Distributed Disk-Based Vector Search with BatANN. arXiv:2512.09331.
- [42] Xiangyu Zhi, Meng Chen, Xiao Yan. (2024). Towards Efficient and Scalable Distributed Vector Search with RDMA. arXiv:2507.06653.
- [43] Qian Xu, Feng Zhang, Chengxi Li. (2024). HARMONY: A Scalable Distributed Vector Database for High-Throughput Approximate Nearest Neighbor Search. arXiv:2506.14707.
- [44] Yi Liu, Fei Fang, Chen Qian. (2024). Efficient Vector Search on Disaggregated Memory with d-HNSW. arXiv:2505.11783.
- [45] Fei Fang, Yi Liu, Chen Qian. (2024). d-HNSW: A High-performance Vector Search Engine on Disaggregated Memory. arXiv:2603.13591.

- [46] Artur Borycki. (2026). Attaching Approximate Nearest Neighbor Indexes to Apache Iceberg Snapshots for Compute-Disaggregated Query Engines. arXiv:2606.04196.
- [47] Zhaoheng Li, Wei Ding, Silu Huang. (2024). Cloud-Native Vector Search: A Comprehensive Performance Analysis. arXiv:2511.14748.
- [48] Lan Lu, Peiqi Yin, Isaac Yang. (2024). ScaleGANN: Accelerate Large-Scale ANN Indexing by Cost-effective Cloud GPUs. arXiv:2605.10135.
- [49] Rongxin Cheng, Yifan Peng, Xingda Wei. (2024). Characterizing the Dilemma of Performance and Index Size in Billion-Scale Vector Search and Breaking It with Second-Tier Memory. arXiv:2405.03267.
- [50] Seoyoung Ko, Hyunjeong Shim, Wanju Doh. (2024). COSMOS: A CXL-Based Full In-Memory System for Approximate Nearest Neighbor Search. arXiv:2505.16096.
- [51] Po-Kai Hsu, Weihong Xu, Qunyou Liu. (2024). HAVEN: High-Bandwidth Flash Augmented Vector ENGINE for Large-Scale Approximate Nearest-Neighbor Search Acceleration. arXiv:2603.01175.
- [52] Liana Patel, Peter Kraft, Carlos Guestrin. (2024). ACORN: Performant and Predicate-Agnostic Search Over Vector Embeddings and Structured Data. arXiv:2403.04871.
- [53] Junjie Song, Yu Liu, Guoyu Hu. (2026). FAVOR: Efficient Filter-Agnostic Vector ANNS Based on Selectivity-Aware Exclusion Distances. arXiv:2605.07770.
- [54] Mingyu Yang, Wenxuan Xia, Wentao Li. (2024). Elastic Index Select for Label-Hybrid Search in Vector Database. arXiv:2505.03212.
- [55] Alireza Heidari, Wei Zhang, Ying Xiong. (2025). FUSEDANN: Convexified Hybrid ANN via Attribute-Vector Fusion. arXiv:2509.19767.
- [56] Alireza Heidari, Wei Zhang. (2025). Filter-Centric Vector Indexing: Geometric Transformation for Efficient Filtered Vector Search. arXiv:2506.15987.
- [57] Joshua Engels, Benjamin Landrum, Shangdi Vu. (2024). Approximate Nearest Neighbor Search with Window Filters. arXiv:2402.00943.
- [58] Jingyi Xi, Chenghao Mo, Benjamin Karsin. (2026). VecFlow: A High-Performance Vector Data Management System for Filtered-Search on GPUs. arXiv:2506.00812.
- [59] Xinkui Zhao, Hengxuan Lou, Yifan Zhang. (2020). GRAB-ANNS: High-Throughput Indexing and Hybrid Search via GPU-Native Bucketing. arXiv:2604.16402.
- [60] Elias Jääsaari, Ville Hyvönen, Teemu Roos. (2025). LEMUR: Learned Multi-Vector Retrieval. arXiv:2601.21853.
- [61] Laxman Dhulipala, Majid Hadian, Rajesh Jayaram. (2024). MUVERA: Multi-Vector Retrieval via Fixed Dimensional Encodings. arXiv:2405.19504.
- [62] Sean MacAvaney, Antonio Mallia, Nicola Tonellotto. (2024). Efficient Constant-Space Multi-Vector Retrieval. arXiv:2504.01818.
- [63] Yao Tian, Zhoujin Tian, Xi Zhao. (2024). GEM: A Native Graph-based Index for Multi-Vector Retrieval. arXiv:2603.20336.

- [64] Silvio Martinico, Franco Maria Nardini, Cosimo Rulli. (2024). Efficient Multivector Retrieval with Token-Aware Clustering and Hierarchical Indexing. arXiv:2604.28142.
- [65] Lixuan Guo, Yifei Wang, Tiansheng Wen. (2025). No More K-means: Single-Stage Sparse Coding for Efficient Multi-Vector Retrieval. arXiv:2605.30120.
- [66] Ximu Zeng, Liwei Deng, Penghao Chen. (2025). LIRA: A Learning-based Query-aware Partition Framework for Large-scale ANN Search. arXiv:2503.23409.
- [67] Taiga Ikeda, Daisuke Miyashita, Jun Deguchi. (2024). ON STORAGE NEURAL NETWORK AUGMENTED APPROXIMATE NEAREST NEIGHBOR SEARCH. arXiv:2501.16375.
- [68] Daichi Amagata, Yusuke Arai, Sumio Fujita. (2024). Learned k-NN Distance Estimation. arXiv:2208.14210.
- [69] Shahrzad Esmat, Chaunté W. Lacewell, Sameh Gobriel. (2024). LLM-Guided ANN Index Optimization for Human-Object Interaction Retrieval. arXiv:2606.05489.
- [70] Xiaoyao Zhong, Jiabao Jin, Peng Cheng. (2024). EnhanceGraph: A Continuously Enhanced Graph-based Index for High-dimensional Approximate Nearest Neighbor Search. arXiv:2506.13144.
- [71] Teodor-Ioan Calin. (2026). Adaptive Prefiltering for High-Dimensional Similarity Search: A Frequency-Aware Approach. arXiv:2602.22214.
- [72] Ruoyang Rykie Guo. (2024). Threshold-Protected Searchable Sharing: Privacy Preserving Aggregated-ANN Search for Collaborative RAG. arXiv:2507.17199.
- [73] Yingfan Liu, Yandi Zhang, Jiadong Xie. (2024). Privacy-Preserving Approximate Nearest Neighbor Search on High-Dimensional Data. arXiv:2508.10373.
- [74] Baiqiang Wang, Qian Lou, Mengxin Zheng. (2024). PIR-RAG: A System for Private Information Retrieval in Retrieval-Augmented Generation. arXiv:2509.21325.
- [75] Hongbin Zhong, Matthew Lentz, Nina Narodytska. (2024). HoneyBee: Efficient Role-based Access Control for Vector Databases via Dynamic Partitioning. arXiv:2505.01538.
- [76] Shanshan Han, Vishal Chakraborty, Sharad Mehrotra. (2024). Don't Be a Pot Stirrer! Authorized Vector Data Retrieval via Access-Aware Indexing. arXiv:2605.01342.
- [77] Dimitris Dimitropoulos, Nikos Mamoulis. (2024). ANN Search: Recall What Matters. arXiv:2606.04522.
- [78] Elias Jääsaari, Ville Hyvönen, Matteo Ceccarello. (2025). VIBE: Vector Index Benchmark for Embeddings. arXiv:2505.17810.
- [79] Mocheng Li, Baotong Lu, Xiao Yan. (2026). Attribute Filtering in Approximate Nearest Neighbor Search: An In-depth Experimental Study. arXiv:2508.16263.
- [80] Wentao Xiao, Yueyang Zhan, Rui Xi. (2024). Enhancing HNSW Index for Real-Time Updates: Addressing Unreachable Points and Performance Degradation. arXiv:2407.07871.
- [81] Shurui Zhong, Dingheng Mo, Siqiang Luo. (2025). LSM-VEC: A Large-Scale Disk-Based System for Dynamic Vector Search. arXiv:2505.17152.
- [82] Patrick Iff, Paul Brügger, Marcin Chrapek. (2024). Benchmarking Filtered Approximate Nearest Neighbor Search Algorithms on Transformer-based Embedding Vectors. arXiv:2507.21989.

- [83] Yitu Wang, Shiyu Li, Qilin Zheng. (2024). NDSEARCH: Accelerating Graph-Traversal-Based Approximate Nearest Neighbor Search through Near Data Processing. arXiv:2312.03141.
- [84] Saroj Mishra, Suman Niroula, Umesh Yadav. (2024). SoK: Agentic Retrieval-Augmented Generation (RAG): Taxonomy, Architectures, Evaluation, and Research Directions. arXiv:2603.07379.
- [85] Aditi Singh, Abul Ehtesham, Saket Kumar. (2024). AGENTIC RETRIEVAL-AUGMENTED GENERATION: A SURVEY ON AGENTIC RAG. arXiv:2501.09136.
- [86] Kairui Fu, Tao Zhang, Shuwen Xiao. (2024). FORGE: Forming Semantic Identifiers for Generative Retrieval in Industrial Datasets. arXiv:2509.20904.
- [87] Yingchen Zhang, Ruqing Zhang, Jiafeng Guo. (2025). C2T-ID: Converting Semantic Codebooks to Textual Document Identifiers for Generative Search. arXiv:2510.19221.